

Resolución del Práctico 2

Testing de Software 2024

Índice

Introducción	3
1. Ejercicio 1	3
2. Ejercicio 2	3
2.1. Métodos cubiertos	3
2.2. Archivos agregados	3
2.3. Estrategia de parametrización	3
2.4. Ejecución	4
2.5. Resultado esperado	4
3. Ejercicio 3	4
3.1. Archivos trabajados	4
3.2. Cobertura de pruebas	4
3.3. Cambios en la implementación	5
3.4. Inconvenientes encontrados	5
3.5. Ejecución	5
4. Ejercicio 4	6
4.1. Archivos	6
4.2. Cobertura de casos	6
4.3. Ejecución	6
5. Ejercicio 5	6
5.1. Archivos	7
5.2. Cobertura de pruebas	7
5.3. Fallas detectadas antes de corregir	7
5.4. Causa del defecto	7

5.5. Corrección aplicada	7
5.6. Ejecución y resultado	7
6. Ejercicio 6	8
6.1. Archivos	8
6.2. Invariante validado por <code>repOK</code>	8
6.3. Tests parametrizados	8
6.4. Ejecución	8
Conclusiones	9

Introducción

Este documento presenta la resolución integral del *Práctico 2*. Se mantiene el contenido técnico original y se mejora su presentación con una estructura académica uniforme, redacción formal y organización por ejercicio.

1. Ejercicio 1

Lectura del capítulo 3 del libro *Introduction to Software Testing*.

2. Ejercicio 2

Se migraron a tests parametrizados los casos desarrollados en el ejercicio 3 del Práctico 1, ejecutándolos sobre la implementación provista por cátedra:

- `src/main/java/assignment2_exercises/SimpleRoutines.java`

2.1. Métodos cubiertos

- `findLast(int[] x, int y)`
- `lastZero(int[] x)`
- `countPositive(int[] x)`
- `oddOrPos(int[] x)`

2.2. Archivos agregados

- `src/test/java/assignment2_exercises/SimpleRoutinesParameterizedTest.java`
- `src/test/resources/assignment2_exercises/odd_or_pos_cases.csv`

2.3. Estrategia de parametrización

- `@MethodSource` en `lastZero` y `countPositive`.
- `@CsvSource` en `findLast`.
- `@CsvFileSource` en `oddOrPos`.

Con esta combinación se satisface lo solicitado: utilizar métodos proveedores y parámetros en formato CSV.

2.4. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=
    SimpleRoutinesParameterizedTest test
```

Se utilizó `-Djacoco.skip=true` debido a incompatibilidades de la versión JaCoCo del template (0.8.2) con JDK actuales.

2.5. Resultado esperado

La suite ejecuta 10 casos. Sobre la versión defectuosa de `SimpleRoutines` fallan los 4 casos que exponen los defectos conocidos:

- `findLast`: no evalúa el índice 0.
- `lastZero`: retorna el primer cero en lugar del último.
- `countPositive`: cuenta 0 como positivo.
- `oddOrPos`: no contabiliza impares negativos.

3. Ejercicio 3

Se desarrolló una suite de tests para todos los métodos de `StackAr` con JUnit 5, utilizando fixture compartido con `@BeforeEach` e identificando explícitamente las fases *arrange*, *act* y *assert*.

3.1. Archivos trabajados

- `src/main/java/assignment2_exercises/stack/StackAr.java`
- `src/test/java/assignment2_exercises/stack/StackArTest.java`

3.2. Cobertura de pruebas

Se incluyeron pruebas para:

- constructor con capacidad inválida,
- `size`,
- `isEmpty`,
- `isFull`,
- `push`,

- `pop`,
- `top`,
- `makeEmpty`,
- `equals`,
- `hashCode`,
- `toString`,
- `repOk`.

Además, se agregaron tests negativos adicionales, por ejemplo:

- `push` en pila llena,
- `push(null)`,
- `pop` en pila vacía,
- `top` en pila vacía.

3.3. Cambios en la implementación

1. Se completó `repOk()` con las condiciones de validez pedidas.
2. Se corrigió `pop()` para que:
 - devuelva el elemento removido del tope,
 - limpie la celda desapilada (`elems[sp] = null`).
3. Se incorporó validación en `push` para rechazar `null`, en consistencia con el invariante.

3.4. Inconvenientes encontrados

- Para validar estados internos inválidos de `repOk` (por ejemplo, `sp` fuera de rango o huecos internos), la API pública no fue suficiente y se utilizó reflexión en los tests.
- Al implementar `repOk` se detectó una inconsistencia adicional en `pop`: no limpiaba la posición desapilada y devolvía el nuevo tope en vez del elemento removido.

3.5. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=StackArTest
    test
```

4. Ejercicio 4

Se implementaron tests parametrizados en CSV para el método:

- `Min.min(List<? extends T>)`

4.1. Archivos

- `src/test/java/assignment2_exercises/MinCsvParameterizedTest.java`
- `src/test/resources/assignment2_exercises/min_valid_cases.csv`
- `src/test/resources/assignment2_exercises/min_invalid_cases.csv`

4.2. Cobertura de casos

- Casos válidos con enteros: mínimo al inicio, al medio, al final, repetidos, negativos y listas unitarias.
- Casos inválidos desde CSV:
 - lista vacía (`IllegalArgumentException`),
 - lista con `null` en primera posición (`NullPointerException`),
 - lista con `null` en otra posición (`NullPointerException`).
- Casos negativos adicionales:
 - lista `null` (`NullPointerException`),
 - elementos no mutuamente comparables (`ClassCastException`).

4.3. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=
    MinCsvParameterizedTest test
```

5. Ejercicio 5

Se construyeron tests parametrizados para `ZuneBug.currentYear(int days)` y, a partir de los fallos observados, se realizó debugging y corrección del método.

5.1. Archivos

- `src/test/java/assignment2_exercises/ZuneBugParameterizedTest.java`
- `src/main/java/assignment2_exercises/ZuneBug.java`
- `ejercicio5/currentYear_original.java.txt` (versión previa a la corrección)

5.2. Cobertura de pruebas

Se agregaron dos grupos parametrizados:

- casos de borde (por ejemplo, 365, 366, 367, 731, 1461),
- múltiples valores para comparar `currentYear` con una función *oracle*.

También se utilizó `assertTimeoutPreemptively` para detectar cuelgues.

5.3. Fallas detectadas antes de corregir

- Con `days = 366`, el método podía entrar en bucle infinito.
- En cambios exactos de año (por ejemplo, 731 y 1461), devolvía un año menos.

5.4. Causa del defecto

La lógica original mezclaba condiciones `>365` y `>366`, con tratamiento incorrecto de límites exactos.

5.5. Corrección aplicada

Se reescribió el bucle utilizando `daysInYear` del año actual:

- si `days >= daysInYear`, se descuenta ese año y se avanza al siguiente,
- en caso contrario, se finaliza y se retorna el año actual.

Esta corrección elimina tanto el bucle infinito como el error *off-by-one* en los bordes.

5.6. Ejecución y resultado

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=
    ZuneBugParameterizedTest test
```

Resultado reportado luego de corregir:

- Tests run: 21, Failures: 0, Errors: 0

- BUILD SUCCESS

6. Ejercicio 6

Se implementó `repOK()` en `BoundedQueue` y se diseñaron tests parametrizados para verificar que el invariante de representación se mantenga luego de operaciones fundamentales de cola.

6.1. Archivos

- `src/main/java/assignment2_exercises/queue/BoundedQueue.java`
- `src/test/java/assignment2_exercises/queue/BoundedQueueParameterizedTest.java`

6.2. Invariante validado por `repOK`

La implementación verifica, entre otros aspectos:

- consistencia estructural básica (`elements`, `capacity`, `size`),
- rangos válidos de `front` y `back`,
- relación de cola circular: `back == (front + size) % capacity`,
- ausencia de `null` en la zona ocupada,
- ausencia de basura fuera de la zona ocupada (celdas en `null`).

6.3. Tests parametrizados

Se utilizó `@MethodSource` con dos grupos:

- escenarios válidos: secuencias de `enqueue/dequeue` (incluyendo *wrap-around*) y chequeo de `repOK` tras cada operación,
- escenarios inválidos: operaciones que deben lanzar excepción (`dequeue` en vacía, `enqueue(null)`, `enqueue` en llena) y verificación de que `repOK` permanezca en `true` después del error.

6.4. Ejecución

```
mvn -Dmaven.repo.local=.m2 -Djacoco.skip=true -Dtest=
    BoundedQueueParameterizedTest test
```

Conclusiones

La resolución del Práctico 2 consolidó el uso de pruebas parametrizadas, análisis de invariantes de representación y depuración de defectos reales. Los ejercicios permitieron integrar técnicas de diseño de tests con verificación de comportamiento funcional, validación estructural y corrección de implementaciones en Java.